# Module Test 5 Preparation Guide

DA3452 · Speech and Language Technology
Portion: after Module Test 4 through LoRA, including the Block 2 worked example.
26 October 2026

This guide maps the announced topics to the textbook and to focused self-study sources. It does not add the rest of Chapters 8 or 9 to the portion. The Block 1 and Block 2 course material supplies the worked examples and extensions beyond J&M.

## Reference convention

J&M means Jurafsky and Martin, Speech and Language Processing, third-edition draft dated **19 August 2026**, supplied for this course. All page numbers below are **printed page numbers**; in this PDF the viewer page is printed page + 8. Use section titles if another draft has different numbering.

| Topic | J&M reading boundary | Course location |
|---|---|---|
| Transformer families and BERT | §§9.1-9.3, pp. 227-235; §9.4 introduction; §9.4.1 and Fig. 9.6, pp. 235-237. Exclude §9.4.2. | Delivered BERT / sequence-classification material |
| Foundation models and task reuse | §1.2, pp. 8-11; §9.4.1; Ch. 7 Historical Notes, p. 207. | Block 1 ML1 and output-head review |
| Prompting and ICL | §1.7 including §1.7.1, pp. 21-24; Fig. 1.14. | Block 1 ML3-ML5 |
| Task head and vocabulary head | §7.5, pp. 193-195; §9.4.1, Eq. 9.6. | Block 1 output-head review |
| Prompt evaluation | §1.9, pp. 25-29; §§4.10-4.11, pp. 115-118, as supporting background. | Block 1 ML6 protocol and statistics |
| Pretraining, loss and scale | §7.7 including §7.7.1, pp. 201-203; §7.8, pp. 204-205. Scaling fits: R3-R4. | Block 1 ML5, ML7-ML8 |
| Instruction tuning and its data | §8.1, pp. 210-213; §§8.1.1-8.1.2; Figs. 8.1-8.5. | Block 2 ML1-ML2 |
| Full fine-tuning, PEFT and LoRA | §8.2, pp. 213-215; Eq. 8.1; Figs. 8.6-8.8. Stop before §8.3 on p. 215. | Block 2 ML3-ML5 |

**Outside the portion:** preference learning, reward models, RLHF, DPO and subsequent material; the explicitly optional Block 1 research aside on ICL mechanisms. Chapters/sections named only as background do not extend the test boundary.

# 1. BERT, reuse and prompting

## Transformer families and contextual representations

• Encoder-only, decoder-only and encoder-decoder families: information flow, causal versus bidirectional self-attention, and the role of the output interface. Encoder-decoder coverage is the family-level comparison already taught.

• Token embeddings versus contextual hidden representations; how context changes the representation of a token. BERT as a bidirectional encoder, rather than an ordinary left-to-right text generator.

• BERT inputs and special tokens, including [CLS] and [SEP]; token, position and segment representations where used. Track sequence length and model dimension in the configurations covered.

**Textbook:** §§9.1-9.2, pp. 227-230. **Paper support:** R1.

## Masked-language-model training and NSP

• Select 15% of eligible tokens in the original BERT setup. Of the selected tokens, replace 80% by [MASK], replace 10% by a random token, and keep 10% unchanged. All selected positions remain prediction targets, including unchanged ones.

• Predict the original token and compute cross-entropy over the selected target positions. Separate the fraction selected for prediction from the fraction visibly replaced by [MASK].

• Original BERT jointly trains with MLM and next-sentence-prediction losses. NSP distinguishes a following segment from a randomly sampled segment; it is not a logical-entailment label.

• BERT versus RoBERTa training objectives and the training regimes covered: RoBERTa drops NSP. Read the broader multilingual discussion only to the depth taught.

**Textbook:** §9.3.1, pp. 231-233; §9.3.2, pp. 233-234; §9.3.3, pp. 234-235. **Paper support:** R1 and R10.

## Task reuse and output heads

A foundation model is broadly pretrained and reusable across downstream tasks. Distinguish a task-specific classifier, a pretrained encoder with a classification head, and a prompted generative language model. Broad reuse does not imply equal accuracy on every task or that a base model is already a chat assistant.

Sequence classification: $\hat{y} = \text{softmax}(h_{CLS}^{L} W_C)$, with $W_C$: $d \times K$.
Vocabulary logits in the tied-weight causal example: $hE^T$, with E: $|V| \times d$.

A task head scores task labels; a vocabulary head scores tokens. A generated label may contain several tokens. Which head is used and which parameters are updated are separate choices. For BERT, the MLM head includes a transform before vocabulary projection.

## Prompting and in-context learning

Identify instruction, completed demonstrations, new query and generated continuation. Zero-shot supplies no demonstrations; one-shot supplies one; few-shot supplies a small set. The unfinished query is not a demonstration. Ordinary ICL changes context, activations and output probabilities while parameters stay fixed. Fine-tuning changes selected parameters. Pretraining can reward context use through next-token loss; this is a motivation, not a complete explanation of ICL mechanisms.

**Textbook:** §1.2; §1.7 and Fig. 1.14; §7.5; §7.7; §9.4.1. **Course:** Block 1 ML1, review, ML3-ML5. **Papers:** R2 and R11.

# 2. Evaluation, scaling and compute

## Evaluating prompts and demonstrations

Separate training/demonstration data, development data for prompt selection, and the final test. Fix model version, inputs, decoding and scoring. Change one factor at a time to study instruction wording, example choice or order.

Distinguish format compliance from task correctness; fix parsing before comparison and report invalid outputs separately. Account for generation variability and finite-test-set uncertainty. Selecting or revising prompts on final-test results biases the reported winner.

Estimated standard error of one accuracy: $SE \approx \sqrt{[\hat{p}(1 - \hat{p})/n]}$.

Approximate 95% interval, when appropriate: $\hat{p} \pm 1.96$ SE.

These describe one accuracy estimate. Two prompts on the same inputs give paired predictions; this interval is not a threshold for their difference. Retain paired bootstrap and McNemar comparison at the depth in Block 1 ML6; no further derivations are added.

**Textbook:** §1.9; §§4.10-4.11 provide background. **Self-study source:** Block 1 ML6 contains the actual comparison protocol, uncertainty example and selection-bias simulation.

## Model size, data and compute

| Symbol | Meaning and distinction |
|---|---|
| N | Model parameter count; state whether embeddings are counted. |
| D | Training-token presentations, including repeated presentations; not necessarily distinct text. |
| C | Training computation in FLOPs; not seconds, bytes or monetary cost. |
| L | Mean next-token cross-entropy; nats per token with natural logarithms. Perplexity = exp(L). |

Dense-model training estimate: $C \approx 6ND$.

Rounded Chinchilla allocation: $D \approx 20N \Rightarrow C \approx 120N^2$.

Hence $N \approx \sqrt{(C/120)}$, $D \approx 20\sqrt{(C/120)}$.

The rough factor six counts about 2N FLOPs per token for the main forward matrix computations and 4N for backward computation. It omits terms and overheads.

## Scaling curves and budget allocation

Interpret logarithmic axes, power-law slopes, diminishing absolute gains and the assumptions of a fitted relation. An IsoFLOP curve fixes compute while trading parameters against tokens; its minimum identifies the best measured allocation for that budget. Equal ND does not force equal loss. Compare loss only with compatible tokenizers, data distributions and loss definitions.

For the Gopher-Chinchilla comparison, retain the parameter/data trade-off, five-shot benchmark protocol and distinction between percentage points and relative percentage change. It is an empirical comparison with other training differences. $D \approx 20N$ is a rounded training-only rule, not a universal prescription.

Rough inference cost for T processed tokens: $C_{infer} \approx 2NT$.

Equating 2NT and 6ND gives $T \approx 3D$, under the same approximations.

T includes prompt and generated tokens. Lifetime inference cost can favour another allocation. Attention/context costs, counting conventions and deployment constraints matter.

**Textbook:** §7.7 including §7.7.1; §7.8; §1.9.1. **Course:** Block 1 ML7-ML8. **Focused self-study:** R3-R4.

# 3. Supervised adaptation and its cost

## Supervised fine-tuning and instruction tuning

Supervised fine-tuning continues training from pretrained parameters on labelled input-target pairs. Instruction tuning is the form that uses instructions and target responses, usually across tasks. Distinguish this from ordinary fixed-weight prompting and from broad next-token pretraining. An instruction-tuned model can still receive demonstrations at inference.

For prompt x and response $y = (y_1,\ldots,y_m)$:
$L_{response} = -\Sigma_{t=1\ldots m} \ln P_\theta(y_t \mid x, y_{<t})$.

Use the course convention: response targets contribute to instruction-SFT loss; prompt and padding targets do not. Prompt tokens still provide context and influence response predictions. State whether the displayed loss is a sum or a mean. Loss masking differs from a causal attention mask and a padding-attention mask.

**Textbook:** §8.1, pp. 210-211; Figs. 8.1-8.2. **Course:** Block 2 ML1.

## Instruction data and evaluation

• Three sources: human-written pairs; existing NLP datasets converted with templates; model-generated pairs. Know how task fields and labels become an instruction-response instance (Figs. 8.3-8.4).

• Distillation transfers behaviour through a teacher’s outputs. In context distillation, a teacher receives additional guidance in its context; the student learns from the resulting responses so the behaviour need not depend on repeating that full guidance.

• Human review, coverage and data quality matter. Fluent imitation can copy style without matching factual accuracy. Recursive training on generated data can lose rare cases and degrade the distribution under the studied conditions; this does not establish that all synthetic-data training fails.

• To evaluate generalisation to unseen task types, withhold a whole task cluster. Withholding one entailment dataset while training on closely related entailment datasets is a weaker separation.

**Textbook:** §8.1.1, pp. 211-213; §8.1.2, p. 213; Figs. 8.2-8.5. **Course:** Block 2 ML2. **Self-study support:** R5-R8; R12 for the original cluster-holdout method.

## Full fine-tuning, frozen parameters and PEFT

Full fine-tuning updates all model parameters. PEFT trains a small selected subset or added parameters while freezing most base parameters. Frozen weights still affect forward computation and can transmit gradients to earlier trainable components. Reducing trainable parameters does not eliminate the base-model forward pass, all activations or all backward computation.

| Stored value per trainable parameter | FP32 bytes |
|---|---|
| Weight | 4 |
| Gradient | 4 |
| Adam first and second moments | 8 |
| Total in the course accounting | 16 |

Full FP32 Adam training state: approximately 16N bytes.
Weights alone: 4N bytes. For $7 \times 10^9$ parameters: 112 GB versus 28 GB.

Use decimal GB ($10^9$ bytes). Activations and other overheads are excluded. Mixed precision and alternative optimizers change this accounting.

**Textbook:** §8.2, pp. 213-214; Fig. 8.6. **Course:** Block 2 ML3. **Paper background:** R13 (Adam).

# 4. LoRA, including the worked example

## Shapes, rank and parameter counts

Use the **J&M / Block 2 row-vector convention**. $W_0$ is the frozen base matrix; only A and B are trained in the layer considered. The course example omits the optional scaling factor.

$x$: $1 \times k$; $W_0$: $k \times d$; $A$: $k \times r$; $B$: $r \times d$.

$\Delta W = AB$; $h = xW_0 + xAB$; $u = xA$.

$\text{rank}(AB) \leq r$; trained entries $= r(k + d)$, versus $kd$ for a full update.

Rank counts linearly independent rows, equivalently columns. A low-rank update can change many entries. Its rank bound does not imply that $W_0 + AB$ has low rank. Count trainable parameters over all adapted matrices; the frozen base weights still require storage. A $4096 \times 4096$ matrix with $r = 8$ trains 65,536 factor entries rather than 16,777,216 entries: a ratio of 1/256.

## Forward computation and backpropagation

Track the base branch $xW_0$, bottleneck $u = xA$, adapter branch $uB$ and output sum. Given the upstream row gradient $g = \partial L/\partial h$, apply the product rule, sum rule and shared-input rule:

$\nabla_B L = u^T g$ $(r \times d)$

$\partial L/\partial u = gB^T$ $(1 \times r)$

$\nabla_A L = x^T(gB^T)$ $(k \times r)$

$\partial L/\partial x = gW_0^T + gB^T A^T$ $(1 \times k)$

The derivative with respect to $W_0$ would be $x^T g$, but it need not be stored or used when $W_0$ is frozen. Its contribution $gW_0^T$ to the input gradient can still be required. Freezing a weight is not the same as blocking gradient flow through the layer.

$A_{new} = A - \eta \nabla_A L$; $B_{new} = B - \eta \nabla_B L$.

Compute both gradients from the same pre-update factors, then apply the step. The Block 2 update is a simple gradient step; the memory discussion uses Adam. Do not replace the worked update with an unspecified Adam step.

## Worked calculation boundary

The Block 2 ML5 example uses $k = 4$, $d = 3$, $r = 1$; $x = [1, 0, -1, 2]$; $A = [1, 2, 0, -1]^T$; $B = [2, -1, 3]$. It gives $xW_0 = [3, 2, 1]$, $xA = -1$, $xAB = [-2, 1, -3]$, and $h = [1, 3, -2]$. The portion includes the supplied upstream-gradient calculation, input gradient, simultaneous factor update, and recomputed output. Understand the method rather than memorising only these outputs.

## Merging and paper notation

$W_{merged} = W_0 + AB \Rightarrow xW_{merged} = xW_0 + xAB$.

For the linear adapter considered, merging removes the extra adapter computation at inference. Swapping adapters requires the matching base weights and configuration. The original paper uses column vectors and $W_0 + B_{paper}A_{paper}$. Under transposition, $A_{course} = A_{paper}^T$ and $B_{course} = B_{paper}^T$. Track shapes rather than exchanging the letters blindly.

**Textbook:** §8.2, pp. 214-215; Eq. 8.1; Figs. 8.7-8.8. **Course:** Block 2 ML4-ML5 is the source for the numerical example and backpropagation. **Paper:** R9. Initialization, scaling-factor choices, rank ablations and LoRA variants are optional extensions unless separately assigned.

# 5. Focused paper reading and self-study

These targets support the already listed portion. Read the named passages for the stated concept; the full paper, unrelated experiments and appendices are **optional**. Paper details do not create new test topics. The course examples remain the reference for the prompt-evaluation protocol, FP32 memory accounting and LoRA arithmetic.

**[R1] Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.**
Open source · §3 overview; §§3.1-3.2; Figs. 1-2.
Supports bidirectional representations, MLM plus NSP, input structure and adding task heads. Restrict downstream-task detail to sequence classification already in the portion.

**[R2] Brown et al. (2020). Language Models are Few-Shot Learners.**
Open source · §1, Figs. 1.1-1.2; opening of §2 and Fig. 2.1.
Supports zero-, one- and few-shot protocols, pretraining versus in-context task use, and the figures used in Block 1. The schematic is not evidence that its exact examples occurred in the training corpus.

**[R3] Kaplan et al. (2020). Scaling Laws for Neural Language Models.**
Open source · Fig. 1; §2.1 and Table 1; §3.2.
Self-study for model-size scaling, logarithmic axes, the non-embedding parameter convention, and the rough compute estimate. §6 is optional background for the historical allocation comparison; no full scaling-law derivation is added.

**[R4] Hoffmann et al. (2022). Training Compute-Optimal Large Language Models.**
Open source · §§3.1-3.4, especially §3.2 and Fig. 3; Tables 2-3; §4 and Table 6.
Self-study for IsoFLOP curves, approximately equal parameter/data scaling, the rounded 20-token rule and the Gopher-Chinchilla comparison. Interpret results and assumptions; fitting procedures and appendix derivations are optional. Appendix F is only a reference for the FLOP-counting caveat.

**[R5] Askell et al. (2021). A General Language Assistant as a Laboratory for Alignment.**
Open source · §2.1, “Context Distillation”.
Self-study for transferring behaviour induced by a long teacher context into student parameters. Understand the teacher-context/student-training distinction. The KL formulation is supporting detail; preference-model training elsewhere in this paper is outside this test.

**[R6] Gudibande et al. (ICLR 2024; preprint 2023). The False Promise of Imitating Proprietary LLMs.**
Open source · Abstract and §1; consult the evaluation discussion as needed.
Self-study for the distinction between copying response style and transferring factual/task performance. Use this as evidence about the studied imitation setups, rather than a universal impossibility claim.

**[R7] Shumailov et al. (2024). AI models collapse when trained on recursively generated data. Nature 631, 755-759.**
Open source · Abstract and introductory explanation of recursive training.
Self-study for model collapse and loss of rare cases under the studied recursive-data conditions. The public precursor is linked on the next page. Proofs, distributional calculations and full experiment details are optional.

# 6. Remaining references and scope boundary

**[R8] Zhou et al. (2023). LIMA: Less Is More for Alignment.**
Open source · Abstract; §2 on the curated training data.
Supports the Block 2 data-quality example. Retain the role of 1,000 curated instruction-response pairs; this does not establish that 1,000 examples suffice for every model or task.

**[R9] Hu et al. (ICLR 2022; preprint 2021). LoRA: Low-Rank Adaptation of Large Language Models.**
Open source · Version 2, §§4.1-4.2 and Fig. 1.
Self-study for the factorised update, frozen base, adapted matrices and merging. Use the course row-vector convention in calculations. Initialization and α/r scaling explain differences from the paper, but are optional extensions to the worked course example.

## Additional source references

**[R10] Liu et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach.**
Open source · §§4.1-4.2 for masking and NSP; opening of §5 for the combined recipe.
Optional original-source detail for the BERT/RoBERTa comparison. The in-scope comparison is the treatment in J&M §9.3.3 and the delivered material.

**[R11] Bommasani et al. (2021). On the Opportunities and Risks of Foundation Models.**
Open source · §1.
Optional background for the broad-pretraining-and-reuse definition. The full report is not assigned for this test.

**[R12] Wei et al. (ICLR 2022; preprint 2021). Finetuned Language Models Are Zero-Shot Learners.**
Open source · §2, including task clusters and evaluation splits.
Original-source support for instruction tuning and withheld task types. Use J&M §8.1.2 and Block 2 ML2 for the required conceptual boundary; the full evaluation suite is optional.

**[R13] Kingma and Ba (ICLR 2015). Adam: A Method for Stochastic Optimization.**
Open source · §2 and Algorithm 1.
Optional optimizer background: identify first- and second-moment state. The test-preparation memory accounting is the explicitly stated FP32 setup in Block 2, not a new assignment to derive Adam.

**Other sources already used in Block 2:** Taori et al. (2023), Alpaca project report for model-generated instruction pairs. Shumailov et al., public precursor, titled The Curse of Recursion: Training on Generated Data Makes Models Forget, for access to the model-collapse study.

## Final boundary

**Included:** the announced BERT/sequence-classification topics; Block 1 core and output-head review; Block 2 through ML5, including LoRA gradients, one update and merging. Required preparation includes the in-scope explanations and calculations that extend beyond J&M.

**Not added by this guide:** full sequence-pair/NER treatments, full-paper experimental details, advanced ICL mechanisms, preference optimisation, RLHF/DPO, or later retrieval and speech material. Merely appearing in a cited paper or a chapter summary does not make a topic part of this test. No new coding task is assigned here.